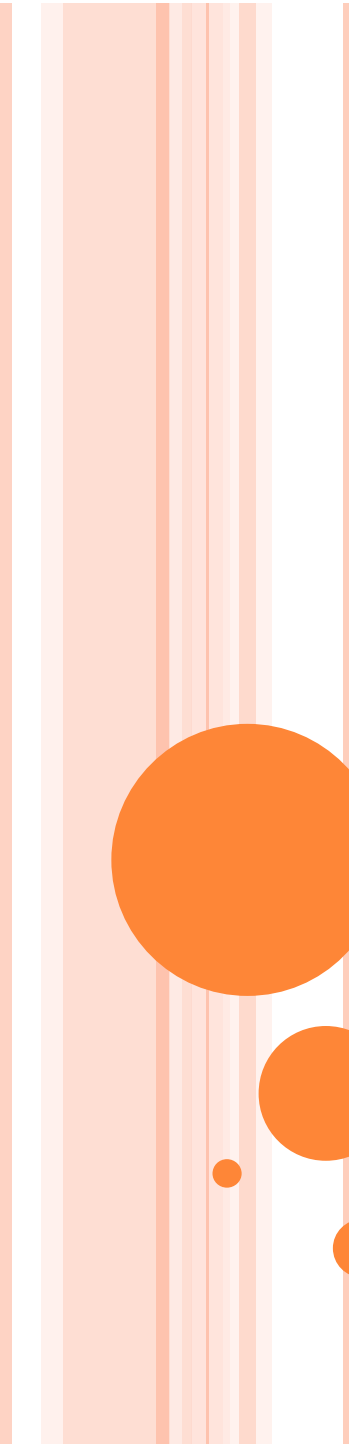
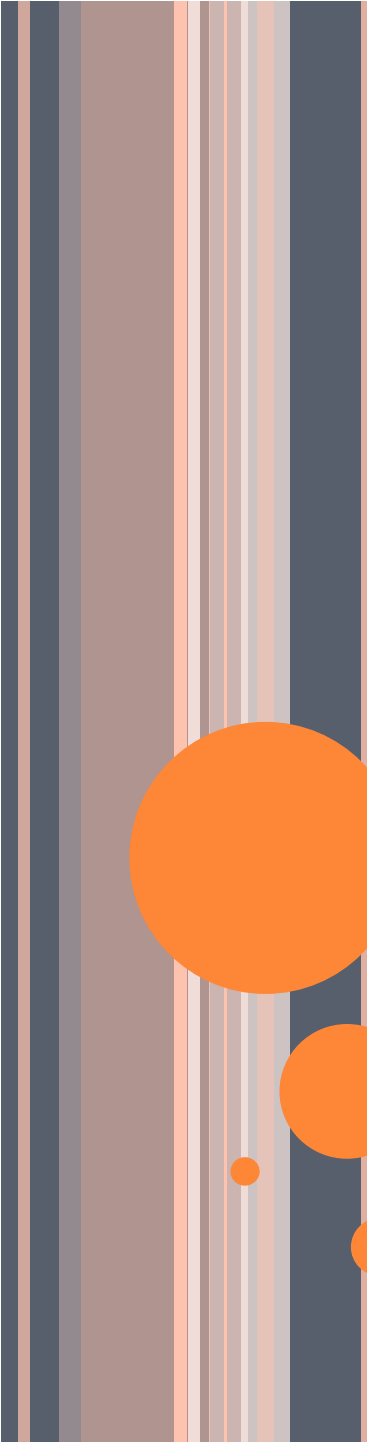


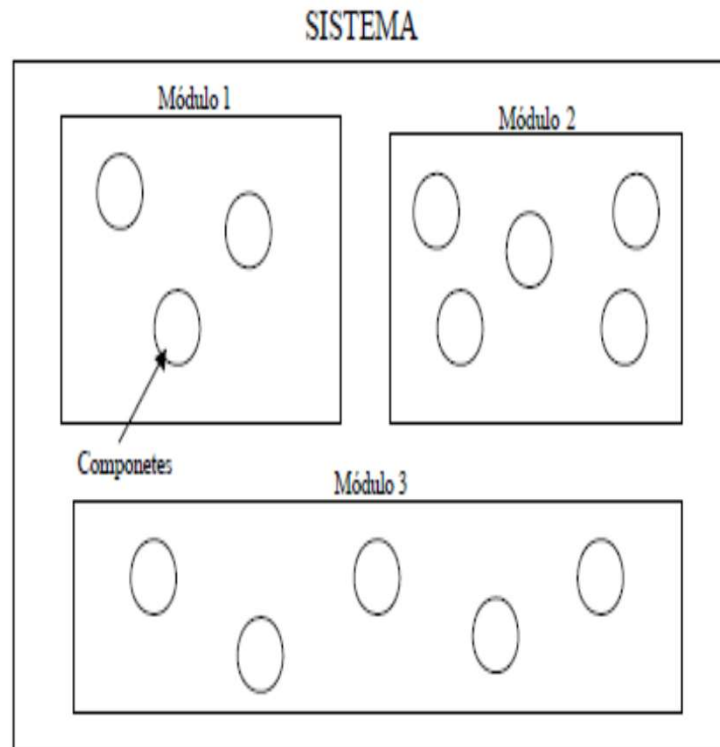


Sistemas de Tiempo Real

Una mirada practica



Diseño con la metodología HRT-HOOD



Descomposición de un sistema

MODULO

cohesión

acoplamiento

- La **descomposición** consiste en dividir un sistema complejo en módulos cada vez más pequeños y aislados, de forma que puedan ser abordados por un único ingeniero o un grupo reducido.
- La **abstracción** permite posponer para más adelante las consideraciones de detalle, especialmente las correspondientes a la implementación. Esto permite una visión simplificada del sistema y de los objetos que contiene, los cuales contienen las propiedades y características esenciales.
- En la descomposición es importante que cada módulo forme una unidad independiente lo más aislada posible del resto de componentes (encapsulación). Existen dos criterios para obtener una buena encapsulación: la cohesión y el acoplamiento.

La **cohesión** hace referencia a la conexión interna de los componentes de un módulo. Existen seis valores (según Allworth y Zobel - 1987) en la medida de la cohesión, que va desde muy poca hasta mucha:

- **casual**: los elementos de un módulo no están ligados entre sí más que de forma superficial, como por ejemplo que se han escrito en el mismo mes.
- **lógica**: los elementos de un módulo están relacionados entre sí en relación al sistema en general, y no en torno al software del proyecto actual, como por ejemplo los drivers de entrada / salida de los periféricos.
- **temporal**: los elementos del módulo se ejecutan en instantes similares, como por ejemplo las rutinas de puesta en marcha.
- **procesal**: los elementos del módulos se utilizan en la misma sección del programa, como por ejemplo los componentes de interface de usuario.
- **comunicacional**: los elementos del módulos funcionan sobre la misma estructura de datos, como por ejemplo los algoritmos utilizados para analizar una señal de entrada.
- **funcional**: los elementos del módulo se utilizan conjuntamente para realizar una determinada función del sistema, como por ejemplo el servir un sistema de ficheros distribuido.

El **acoplamiento**, por el contrario, es una medida de la interdependencia entre los módulos de un programa. Si dos módulos se pasan información de control entre ellos, se dice que tienen un grado bajo de acoplamiento. Y al contrario, el acoplamiento es alto si únicamente se pasan datos. Otra forma de saber si el acoplamiento de un módulo con el resto es bajo es medir la facilidad de eliminar o reemplazar este módulo por otro alternativo.

En todos los métodos de diseño, una buena descomposición es aquella que tiene una **fuerte cohesión en todos sus módulos** y un **grado de acoplamiento bajo** entre ellos. Este principio es válido tanto en la programación secuencial como en la concurrente.

HRT-HOOD

- Esta metodología tiene en cuenta las necesidades de los sistemas de tiempo real. En ésta el proceso de diseño se ve como una progresión creciente de compromisos específicos.
- Los compromisos definen las propiedades que se deben ir cumpliendo en el diseño del sistema y que el diseñador no puede modificar en niveles de mayor detalle.
- Aquellos aspectos del diseño que no suponen un compromiso en un determinado nivel se deben asumir como obligaciones en los niveles inferiores (mayor detalle) del sistema.
- Las obligaciones en el nivel superior corresponden a las propiedades que aparecen en las especificaciones.
- Los compromisos son las concreciones que se hacen en el diseño que hacen que se cumplan las propiedades correspondientes a las obligaciones.
- El proceso de refinamiento del diseño consiste en ir transformando las obligaciones en compromisos. Este proceso está sujeto a restricciones impuestas principalmente por el entorno de ejecución. El entorno de ejecución es un conjunto de componentes software y hardware sobre los que se construye el sistema.

HRT-HOOD

- Cada uno de estos compromisos se pueden describir con una serie de obligaciones que, a su vez, se podrán concretar en compromisos cada vez más detallados y sencillos.
- En HRT-HOOD se definen dos tipos de actividades en el diseño de la arquitectura:
 - Diseño de la arquitectura lógica.
 - Diseño de la arquitectura física.
- La arquitectura lógica se ocupa de los compromisos que se pueden hacer con independencia de las restricciones impuestas por el entorno de ejecución. Supone principalmente los requerimientos funcionales.
- La arquitectura física tiene en cuenta los requerimientos funcionales y, además, las restricciones del entorno. Abarca además los requerimientos no funcionales. En la arquitectura física es donde se tienen en cuenta las restricciones temporales.

- Si llamamos P1 a la propiedad de poder controlar cuando se ejecutan las operaciones invocadas desde otros objetos, y P2 la propiedad de poder invocar operaciones de otros objetos de forma espontánea, tendremos:

TIPO	P1	P2	COMENTARIO
Pasivo	No	No	Máxima restricción
Activo	Sí	Sí	Sin restricciones
Protegido	Sí	No	Bloqueos controlados
Cíclico	Sí	Sí	Periódicas. Interfaz sencilla
Esporádico	No	Sí	Una única operación

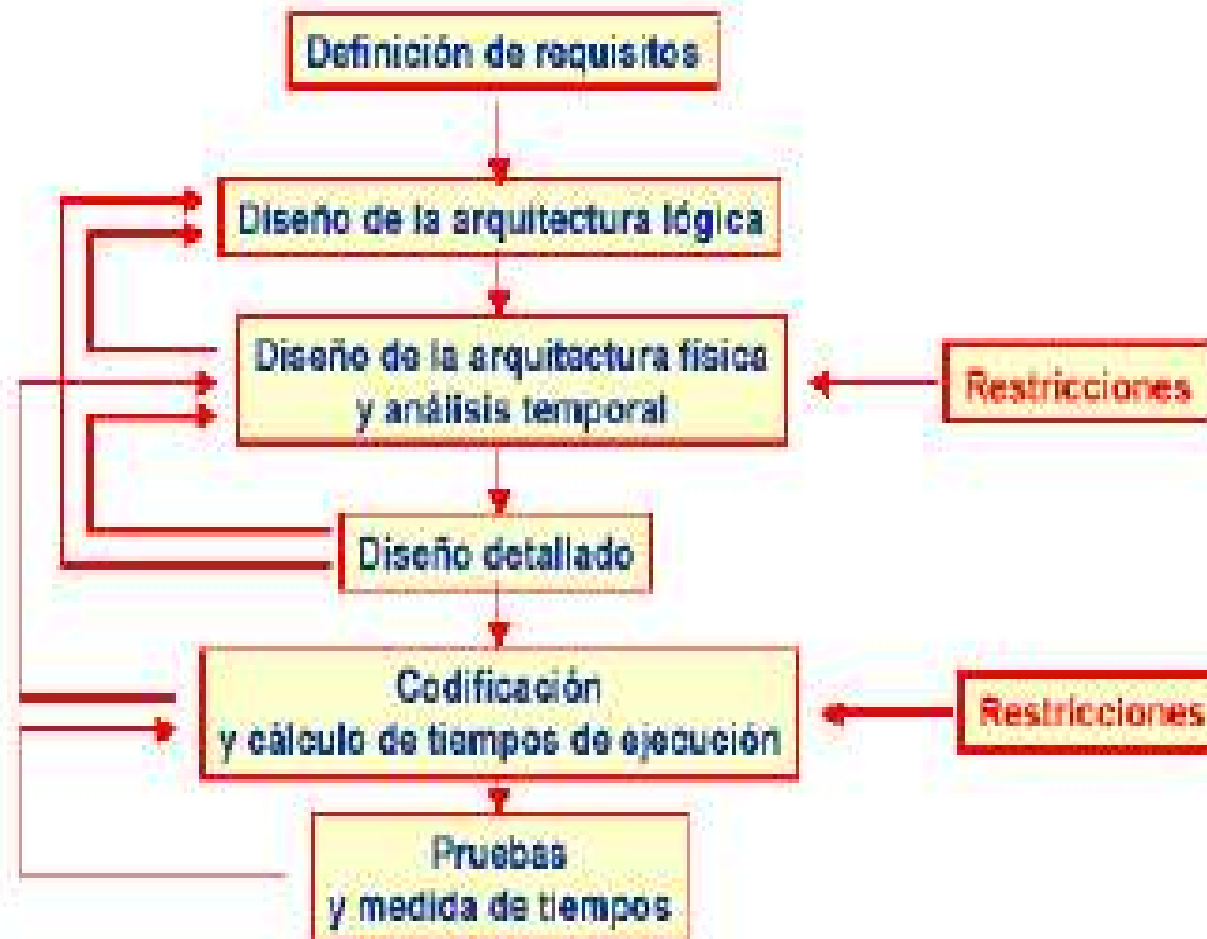
Método de diseño desarrollado en la Universidad de York, orientado a la construcción de sistemas de tiempo real, en particular a los llamados críticos (*hard*). Estos son sistemas que tienen componentes que, si no producen una respuesta dentro de un intervalo de tiempo prefijado, pueden ocasionar daños importantes.

El proceso de refinamiento del diseño, que consiste en transformar las obligaciones (aspectos del diseño que no están sujetos a responsabilidades) en responsabilidades, a menudo se encuentra sujeto a restricciones impuestas por el entorno de ejecución, es decir, el conjunto de componentes hardware y software (procesadores, planificadores, controladores de dispositivos) sobre el que se construye el sistema. Impone restricciones de recursos como la velocidad del procesador, el ancho de banda para las comunicaciones, y restricciones de los mecanismos como la prioridad de las interrupciones, la planificación de tareas y el bloqueo de los datos.

Un método de diseño adecuado para tiempo real debe proporcionar:

- Reconocimiento explícito de los tipos de actividades/objetos que se pueden encontrar en sistemas críticos (por ejemplo, actividades cíclicas y esporádicas).
- Integración de los parámetros apropiados de planificación en el proceso de diseño.
- Definición explícita de los requisitos temporales de cada objeto de la aplicación.
- Definición de la importancia relativa de cada objeto para el funcionamiento correcto de la aplicación.
- Facilidades para realizar el análisis de la planificabilidad del sistema.

Proceso de desarrollo iterativo



Diseño de la arquitectura lógica: Fase destinada fundamentalmente a la satisfacción de los requisitos funcionales. El resultado de esta etapa es un conjunto de objetos terminales (que no requieren mayor descomposición).

Objetivos de la arquitectura física:

- Relacionar la arquitectura lógica con los recursos de ejecución.
- Asignar atributos temporales a los objetos y asegurar el cumplimiento de los requisitos no funcionales:
- Análisis de plazos de respuesta.
- Fiabilidad y seguridad.

En el diseño de la arquitectura física se consideran principalmente 4 actividades:

- Asociación de los objetos resultantes de la arquitectura lógica a los recursos hardware (procesadores) disponibles.
- Planificación de la red. Si el sistema está distribuido, hay que planificar las comunicaciones entre los nodos, de forma que el tiempo de retardo en las comunicaciones esté acotado.
- Planificación de los procesadores. Determinar la planificación, estática o dinámica, que asegure que todas las tareas del sistema cumplan con los plazos prefijados.
- Análisis de la fiabilidad: determinar si es conveniente emplear técnicas de tolerancia de fallos, para aumentar la fiabilidad del sistema.

El resultado del modelo físico es un sistema de objeto terminales con atributos temporales. Aunque la arquitectura física es un refinamiento de la arquitectura lógica, el desarrollo de ambas etapas será iterativo y concurrente.

Diseño con la metodología HRT-HOOD

HRT-HOOD facilita el diseño de la arquitectura lógica de un sistema ofreciendo diferentes tipos de objetos. Estos son:

- **Pasivos:** Objetos reentrantes que no tiene control sobre cuando son invocadas las operaciones que ejecutan, y no invocan de forma espontánea operaciones en otros objetos.
- **Activos:** Objetos que pueden controlar cuando se ejecutan las invocaciones de sus operaciones, y pueden espontáneamente invocar operaciones de otros objetos. Los objetos activos son la clase más general de objetos y no hay restricciones impuestas sobre ellos.
- **Protegidos:** Objetos que pueden controlar cuando se ejecutan las invocaciones de sus operaciones, pero no pueden invocar operaciones de otros objetos de forma espontánea; en general los objetos protegidos no pueden tener restricciones de sincronización arbitrarias y deben ser analizados los tiempos de bloqueos que imponen a sus llamantes.
- **Cíclicos:** Objetos que representan actividades periódicas; pueden invocar espontáneamente operaciones de otros objetos y tienen sólo interfaces muy restrictivos.
- **Esporádicos:** objetos que representan actividades esporádicas; los objetos esporádicos pueden invocar operaciones de otros objetos de forma espontánea; cada uno tiene una operación simple que es llamada cuando es invocado el objeto esporádico.

PASIVOS

- No controlan cuándo se ejecutan sus operaciones. Cuando se invoca una operación, el control se transfiere inmediatamente a esa operación.
- No invocan operaciones de otros objetos espontáneamente.
- Sólo contienen código secuencial.

PROTEGIDOS

- Pueden controlar cuándo se ejecutan sus operaciones.
- No invocan operaciones en otros objetos espontáneamente.
- Se utilizan para controlar el acceso a recursos compartidos por varios objetos activos.

ACTIVOS

- Pueden controlar cuándo se ejecutan sus operaciones.
- Pueden invocar operaciones en otros objetos espontáneamente.

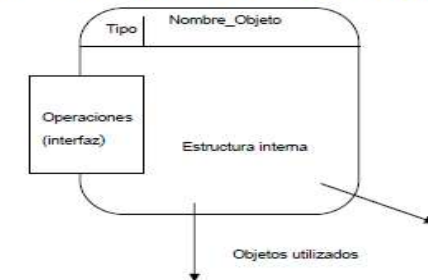
CÍCLICOS

- Representan actividades periódicas.
- Sólo pueden incluir operaciones que demanden atención inmediata (ATC: transferencias asincrónicas de control). Muchos objetos cíclicos no tienen operaciones

ESPORÁDICOS

- Representan actividades esporádicas. Incluyen UNA operación START y pueden incluir una o más ATC.
- Los objetos cíclicos y esporádicos tienen **1 thread** (actividad concurrente) que se ejecuta independientemente de las operaciones.
- Las ATC se utilizan para obtener una atención inmediata del thread.
- **OBCS** (object control structure). Controla la ejecución de las operaciones del objeto..

TIPOS DE OBJETOS EN HRT-HOOD



7.3 ATRIBUTOS DE TIEMPO REAL

Algunos de los principales atributos de un objeto son:

- **DEADLINE.** Cada objeto cíclico y esporádico puede tener un tiempo límite definido para la ejecución de su thread.
- **OPERATION_WCET.** Cada operación visible debe tener definido un tiempo de ejecución máximo.
- **THREAD_WCET.** Cada objeto cíclico y esporádico visible debe tener definido un tiempo de ejecución máximo para su thread.
- **PERIOD.** Cada objeto cíclico debe tener definido un periodo de ejecución.
- **OFFSET.** Cada objeto cíclico puede tener definido un *offset* que indica el tiempo que el thread debe esperar antes de iniciar sus operaciones cíclicas.
- **MINIMUM_ARRIVAL_TIME o MAXIMUM_ARRIVAL_FREQUENCY.** Cada objeto esporádico debe tener definido uno de estos dos atributos, que indican el tiempo mínimo o la frecuencia máxima entre dos peticiones de activación.
- **PRIORITY.** Cada objeto cíclico y esporádico puede tener una prioridad definida para su thread.

7.4.1 TIPOS DE OPERACIONES DE UN OBJETO PROTEGIDO

Un objeto protegido tiene un OBCS. Las operaciones acceden a los datos internos con exclusión mutua. 2 tipos de restricciones en las operaciones de un objeto protegido:

- PSER. *Protected synchronous execution request*. La actividad del objeto cliente se interrumpe hasta que se completa la ejecución de la operación protegida.
- PAER. *Protected asynchronous execution request*. La actividad del cliente se interrumpe sólo hasta que la petición de la operación ha sido recibida por el objeto protegido..

También pueden tener operaciones sin restricciones, similares a las de los objetos pasivos.

7.4.2 TIPOS DE OPERACIONES DE UN OBJETO ACTIVO

- **CONSTRAINED** (Con restricciones). Se ejecutan bajo el control del OBCS.
- **UNCONSTRAINED** (Sin restricciones). Se ejecutan tan pronto son invocadas, similares a las de los pasivos.

Se establecen 2 tipos de restricciones:

- | | | |
|----|--|----|
| a) | Funcionales. Restricciones en función del estado interno del objeto. Operación abierta: el estado interno del objeto permite la ejecución de la operación. Ejemplo: objeto BUFFER. Operación PUT abierta si el estado NO es LLENO. | a) |
| b) | Según tipo de invocación. Indican el efecto sobre el objeto que invoca la operación: | b) |
| • | ASER. Asynchronous Execution Request. Cuando se llama a una operación ASER, el objeto que llama no se bloquea. | • |
| • | LSER. Loosely Synchronous Execution Request. El llamador se bloquea hasta que el objeto llamado está listo para ejecutar la operación. | • |
| • | HSER. Highly Synchronous Execution Request. El llamador se bloquea hasta que el objeto llamado ha atendido la petición (ha ejecutado la operación). | • |

Las operaciones LSER y HSER pueden tener asociado un *timeout*.

7.4.3 OPERACIONES DE UN OBJETO CÍCLICO

En general, los objetos cíclicos se comunicarán y sincronizarán con otros threads mediante llamadas a operaciones de objetos protegidos. Sin embargo, puede ser necesario definir algunas operaciones restringidas sobre objetos cíclicos, porque otros objetos pueden necesitar señalar algún cambio de modo o alguna condición de error al objeto cíclico. Podría conseguirse haciendo que el objeto cíclico haga un *polling* a una operación 'notificadora del cambio de modo' o 'notificadora de error' de un objeto protegido, pero esta solución es ineficiente si el tiempo de respuesta del objeto cíclico es pequeño.

Las operaciones restringidas que se definan en el objeto cíclico requerirán una respuesta inmediata del *thread* del objeto. El OBCS del objeto interaccionará con el *thread* para forzar una ATC. Estas operaciones restringidas suelen denominarse ASATC (*Asynchronous asynchronous transfer of control request*), similares a las ASER de los objetos activos, excepto que demandan que el objeto cíclico reaccione de manera inmediata.

7.4.4 OPERACIONES DE UN OBJETO ESPORÁDICO

Cada objeto esporádico tiene una operación restringida, llamada habitualmente START, que se llama para activar la ejecución del *thread*. Es una operación ASER, y puede ser invocada por una interrupción (ASER_BY_IT).

Un objeto esporádico puede además tener otras operaciones restringidas ASATC, iguales a las mencionadas en los objetos cíclicos.

7.5 REGLAS DE DESCOMPOSICIÓN DE LOS OBJETOS

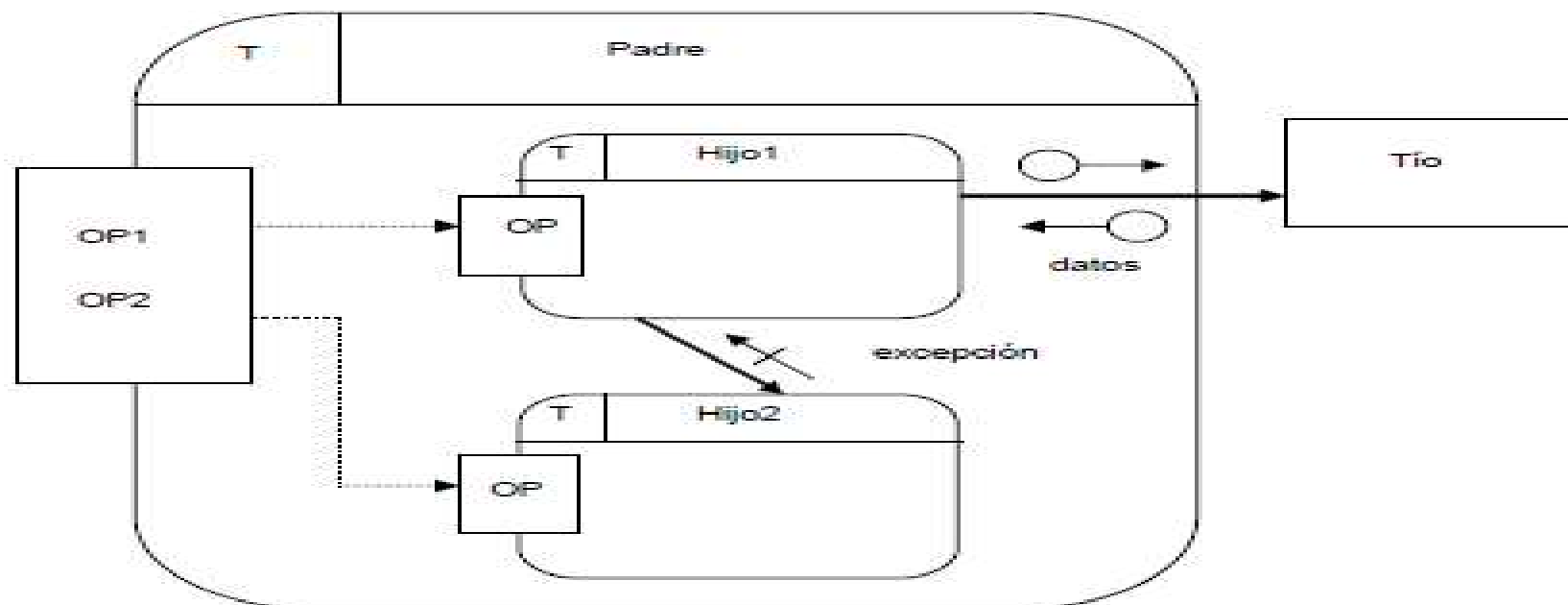
Un objeto (padre) puede incluir uno o más objetos (hijos).

- Un objeto activo puede incluir cualquier objeto. Un objeto activo no puede ser un objeto terminal en HRT-HOOD.
- Un objeto pasivo sólo puede incluir otros objetos pasivos.
- Un objeto protegido puede incluir objetos pasivos y un objeto protegido.
- Un objeto esporádico puede incluir objetos esporádicos, cíclicos, pasivos y protegidos.
- Un objeto cíclico puede incluir objetos cíclicos, esporádicos, pasivos y protegidos.

Una operación restringida de un objeto padre puede ser implementada por una operación restringida de un hijo.

REGLAS DE USO

- Los objetos pasivos no pueden utilizar (invocar) operaciones restringidas de otros objetos.
- Objetos cíclicos y esporádicos pueden utilizar operaciones de objetos pasivos, protegidos, cíclicos y esporádicos. Pueden utilizar operaciones ASER de objetos activos.
- Los objetos protegidos pueden utilizar operaciones de objetos pasivos y protegidos, operaciones ASATC de objetos cíclicos, operaciones START y ASATC de objetos esporádicos y operaciones ASER de objetos activos.



El objeto padre se descompone en Hijo1 e Hijo2

Padre usa Tío

Hijo1 usa Hijo2 y Tío (Hijo1 es cliente de Hijo2. Hijo2 es servidor de Hijo1)

Hijo1.OP implementa Padre.OP1

Hijo2.OP implementa Padre.OP2

CODIFICACIÓN EN ADA95

- Cada objeto se realiza mediante un paquete.
- Las relaciones de uso se representan mediante cláusulas with.
- Los objetos hijos se realizan mediante paquetes hijos privados.
- Los objetos cíclicos y esporádicos tienen una tarea, y posiblemente (los esporádicos obligatoriamente) un objeto protegido de sincronización (para el OBCS).
- Objeto protegido: objeto protegido de Ada95.
- Los atributos temporales de cada objeto se definen en un paquete auxiliar.

```
with Tio;
```

```
package Padre is  
    procedure OP1 (...);  
    procedure OP2 (...);  
end Padre;
```

```
private package Padre.Hijo1 is  
    procedure OP;  
end Padre.Hijo1;
```

```
private package Padre.Hijo2 is  
    procedure OP;  
end Padre.Hijo2;
```

```
with Padre.Hijo1, Padre.Hijo2;  
package body Padre is  
    procedure OP1(...) renames Padre.Hijo1.OP;  
    procedure OP2(...) renames Padre.Hijo2.OP;  
end Padre;
```

Definición del problema

Queremos desarrollar un RTS que comprende:

- a) Una entidad encargada de producir datos a una velocidad constante (p.e., cada 20 ms). PRODUCTOR.
- b) Una entidad encargada de consumir esos datos, según petición del Productor. El Productor y el Consumidor tienen que almacenar un registro sobre su actividad en una estructura de datos dedicada a tal fin. CONSUMIDOR.
- c) Una entidad encargada de transmitir los contenidos de dicha estructura de datos al operador del sistema, bajo petición del Productor. TRANSMISOR.

Estas entidades tendrán que realizar además otras actividades auxiliares en cada activación.

OBJETOS:

BUFFER es un objeto protegido en el que PRODUCTOR almacena datos, y del que CONSUMIDOR los retira. Proporciona 2 operaciones protegidas sincrónicas: Read y Write.

ESTRUCTURA es un objeto protegido en el que PRODUCTOR y CONSUMIDOR almacenan los registros de su actividad, y del que TRANSMISOR los extrae. Proporciona 2 operaciones protegidas sincrónicas: Read y Write.

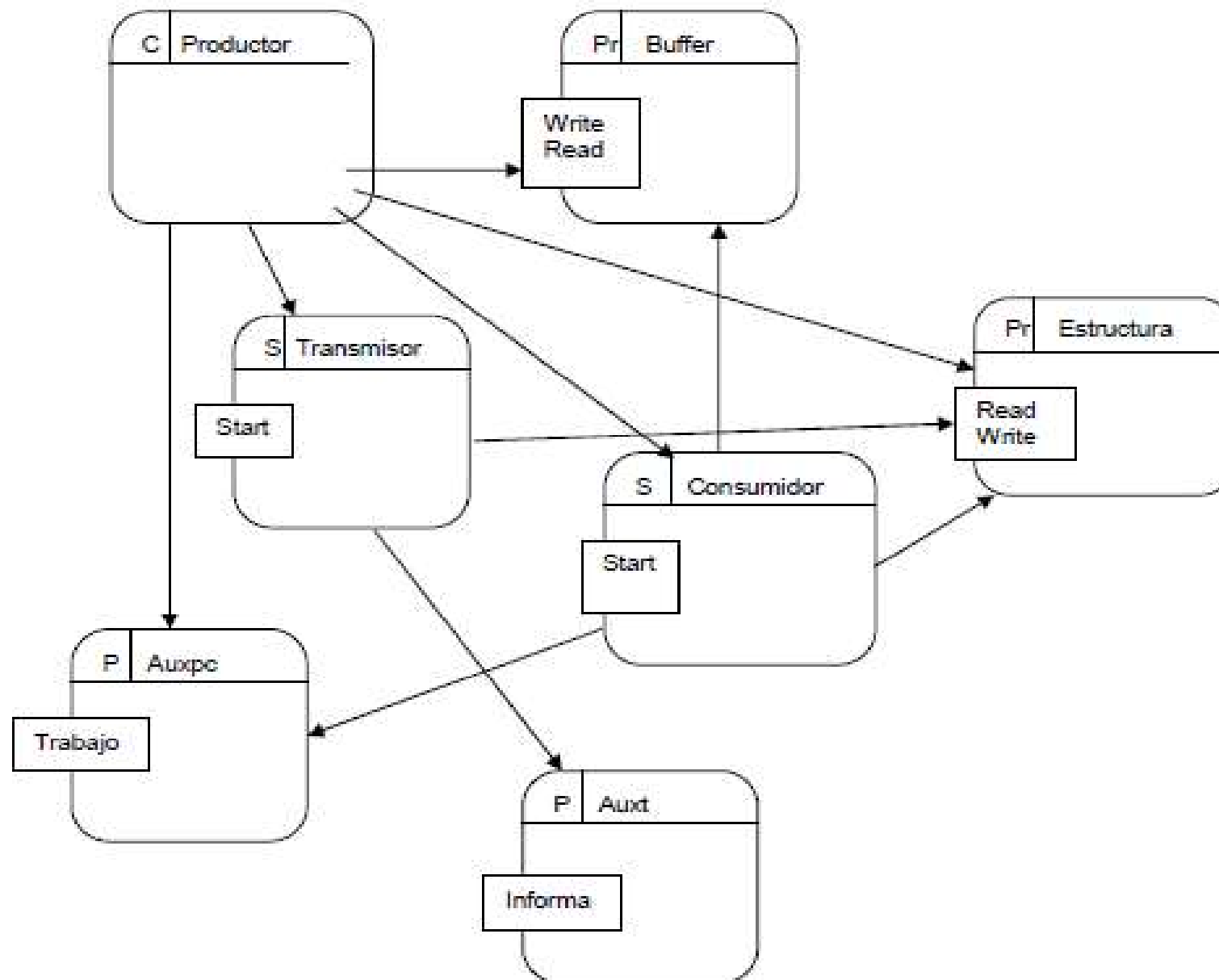
PRODUCTOR es un objeto cíclico. Dispone de 1 thread.

CONSUMIDOR es un objeto esporádico. Dispone de 1 thread y 1 OBCS. Tiene una operación asíncrona Start.

TRANSMISOR: objeto esporádico. Dispone de 1 thread y 1 OBCS.

AUXPC: objeto pasivo que contiene las operaciones auxiliares del PRODUCTOR y del CONSUMIDOR. Le asociamos una operación Trabajo (no restringida).

AUXT: objeto pasivo que contiene las operaciones auxiliares del TRANSMISOR. Le asociamos una operación Informa (no restringida).



Las flechas entre parejas de objetos representan las relaciones de uso.

En cada activación periódica, el objeto PRODUCTOR:

- Deposita un dato en el BUFFER utilizando la operación sincrónica Write de éste.
- Realiza una serie de trabajo auxiliar, llamando a la operación Trabajo de AUXPC.
- Llama a la operación sincrónica Write de ESTRUCTURA para almacenar información que le interese al operador.
- Activa los objetos CONSUMIDOR y TRANSMISOR, llamando a sus operaciones asincrónicas Start.

En cada activación, CONSUMIDOR:

- Lee el dato almacenado en BUFFER, utilizando la operación Read de éste.
- Realiza trabajo auxiliar, llamando a la operación Trabajo de AUXPC.
- Llama a Write de ESTRUCTURA para almacenar información que le interese al operador.

En cada activación, TRANSMISOR:

- Llama a la operación sincrónica Read de ESTRUCTURA, para leer la información sobre el estado del sistema que le va a presentar al operador.
- Llama a la operación Informa de AUXT, que produce la salida de dicha información hacia el operador.

PRODUCTOR, CONSUMIDOR y TRANSMISOR son objetos activos, por lo que tiene cada uno 1 thread de control. Las operaciones de estos threads estarán sujetas a requisitos temporales específicos.

Supongamos los siguientes requisitos:

- PRODUCTOR tiene un periodo de activación (T) de 20ms, y un plazo de respuesta (deadline, D) de 9ms.
- CONSUMIDOR tiene un tiempo mínimo entre activaciones igual al periodo del productor, y un plazo que nos asegure que finalice antes del siguiente ciclo del productor. Por tanto, podemos poner su $D=17\text{ms}$.
- TRANSMISOR tendrá un tiempo entre activaciones igual al del consumidor, y un plazo menor que el del productor ($D=8\text{ms}$).